Software Design Specifications

for

Student Performance Visualization Tool

Prepared by: TEAM - 39

Team members:

E.PRIYAMVADA	SE23UMCS012
M.LEELA KARTHIKEYA	SE23UMCS039
NAVVYAH	SE23UMCS044
M.SAI SATHWIK	SE23UMCS041
A.SATHVIK REDDY	SE23UMCS001
SATYA VARDHAN	SE23UMCS022
NEAL TEJA	SE23UMCS004

Document Information

Title: STUDENT PERFORMANCE VISUALIZATION TOOL	Document Version No: 1.0
Project Manager: Sai Srithaja Nibanupudi	
Prepared By: TEAM – 39	**Preparation Date:** 6 APRIL 2026

Table of Contents

1 Introduction

The Software Design Specification (SDS) for the EduTrack Student Performance System provides a comprehensive overview of the system's design and architecture. This document describes how the system is structured, how different components interact, and how data flows between modules.

The purpose of this document is to clearly present the design of the system so that developers, testers, and evaluators can understand its implementation and functionality. It serves as a guide for system development, maintenance, and future enhancements.

EduTrack is a role-based web system that helps coordinators, teachers, and students manage and visualize academic performance. It combines a Node.js/Express backend, static HTML dashboards, real-time updates with Socket.IO, and dual data stores (MongoDB and MySQL) for efficient access to user profiles and academic records.

1.1 Purpose

The purpose of this Software Design Specification (SDS) document is to provide a detailed description of the design and internal structure of the EduTrack Student Performance System. This document explains how different components of the system interact with each other, how data is processed, and how functionalities are implemented.

The software design of EduTrack to help developers, testers, coordinators, and maintainers understand the system architecture, data model, APIs, and key workflows. It serves as the implementation guide for the current codebase and future enhancements.

1.2 Scope

The EduTrack system is a web-based application developed to manage and analyze student academic performance efficiently. The system supports three categories of users: coordinators, teachers, and students.

Coordinators are responsible for managing student records, including adding new students, updating attendance, and deleting records. Teachers can enter and update student marks, evaluate performance, and monitor class-level progress. Students can log in to view their academic details such as marks, attendance, grades, and graphical performance analysis.

The system is built using modern web technologies such as Node.js and Express.js for backend development. It uses MongoDB for storing user credentials and student profiles, and MySQL for storing structured data such as marks and attendance. Real-time communication is implemented using Socket.IO.

1.3 Definitions, Acronyms, and Abbreviations

Term	Meaning
SDS	Software Design Specification
SPVT	Student Performance Visualization Tool
JWT	JSON Web Token (stateless authentication token)
REST	Representational State Transfer (API style)

ORM	Object Relational Mapping (Sequelize)
UI	User Interface
API	Application Programming Interface
DB	Database

1.4 References

This section provides a list of all documents, resources, and references used in the design and development of the EduTrack Student Performance System. These references include official documentation, technical resources, and project-related materials that support the system design.
The following references were used:

1. **Node.js Official Documentation** Source: https://nodejs.org/en/docs
   Description: Provides information about server-side JavaScript execution, asynchronous programming, and backend development.

2. **Express.js Documentation**
   Source: https://expressjs.com/
   Description: Provides details about building web applications and APIs using Express framework.

3. **MongoDB Documentation**
   Source: https://www.mongodb.com/docs/
   Description: Provides guidance on NoSQL database design, data storage, and query handling.

4. **MySQL Documentation**
   Source: https://dev.mysql.com/doc/
   Description: Provides information on relational database design, SQL queries, and database management.

5. **Socket.IO Documentation**
   Source: https://socket.io/docs/
   Description: Provides details on real-time communication between client and server using WebSockets.

6. **Project Source Code**
   Source: `/Users/karthikeya/Desktop/spvt_1`
   EduTrack Project Files (Controllers, Modules, Models, Routes)
   Description: Includes all backend modules such as authentication, student management, marks processing, validation, and notification.
7. **Software Design Specification Template**
   Source: Course Assignment Material / Instructor Provided Template Description: Used as a guideline for structuring the SDS document.

2 Use Case View

Based on the requirements of the EduTrack Student Performance System, this section presents the major use cases that represent the core functionalities of the system. These use cases describe the interactions between different users and the system and highlight the key operations performed within the application.
The system is designed to support three primary actors: coordinator, teacher, and student. Each actor interacts with the system in different ways based on their roles and permissions. The use cases included in this section cover significant functionalities such as authentication, student management, marks entry, performance visualization, and notification handling.
These use cases are selected because they represent the central behavior of the system and involve multiple design elements such as database interaction, validation, processing logic, and real-time communication.

2.1 Use Case

For each use case in the EduTrack Student Performance System, a detailed description is provided including its name, a brief explanation, and the sequence of steps involved in performing the use case. These use cases are designed based on the functional requirements of the system and illustrate how the system behaves in response to user actions.

Use Case 1: User Login

Brief Description:
This use case describes how a user logs into the system using valid credentials.

Actors: Coordinator, Teacher, Student

Usage Steps / Procedure:
The user enters their username and password on the login interface. The system sends the entered credentials to the backend server. The authentication module verifies the credentials against the stored data in the database. If the credentials are valid, a secure token is generated and the user is granted access to the system. If the credentials are invalid, an error message is displayed and access is denied.

Use Case 2: Add Student

Brief Description:
This use case describes how a coordinator adds a new student to the system.

Actors: Coordinator

Usage Steps / Procedure:
The coordinator navigates to the student management section and enters student details such as student ID, name, and class. The system validates the entered data. If the data is valid, the system stores the student information in the database and initializes related records such as attendance and performance data. The student is then successfully added to the system.

Use Case 3: Update Marks

Brief Description:

This use case describes how teachers or coordinators update student marks.

Actors: Teacher, Coordinator

Usage Steps / Procedure:

The teacher selects a student and enters marks for different subjects. The system validates the marks to ensure they fall within the acceptable range. Once validated, the marks are stored in the database. The system calculates the average marks and assigns a grade. Notifications are generated and sent to relevant users.

Use Case 4: View Performance

Brief Description:

This use case describes how a student views their academic performance.

Actors: Student

Usage Steps / Procedure:

The student logs into the system and accesses the dashboard. The system retrieves marks, attendance, and processed results from the database. The visualization module processes this data and displays it in graphical formats such as charts and graphs.

Use Case 5: Receive Notifications

Brief Description:

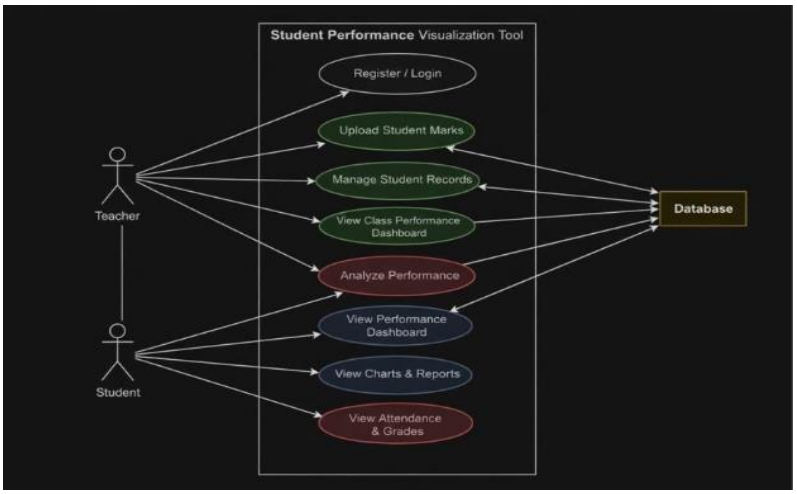
This use case describes how users receive real-time notifications from the system.

Actors: Coordinator, Teacher, Student

Usage Steps / Procedure:

Whenever an important event occurs, such as marks update or attendance change, the system generates a notification. This notification is sent to users through real-time communication. The notification is displayed on the user interface.

Use case diagram:



3 Design Overview

This section provides an overview of the overall software design of the EduTrack Student Performance System. The design follows a modular and layered architecture, ensuring separation of concerns, scalability, and maintainability.

The system is implemented using a client-server model, where the frontend interacts with the backend through RESTful APIs. The backend is responsible for handling business logic, data processing, and database operations.

The design complies with high-level requirements such as secure authentication, efficient data management, real-time notifications, and performance visualization.

The system is divided into multiple modules including authentication, student management, marks processing, validation, notification, and visualization. Each module is designed to perform a specific function and interact with other modules through well-defined interfaces.

3.1 Design Goals and Constraints

The primary goal of the EduTrack system is to provide an efficient and scalable platform for managing student performance data. The design focuses on modularity, security, performance, and real-time communication.

One of the major design goals is to ensure secure access to the system using authentication mechanisms such as JWT tokens. Another goal is to provide real-time updates to users using WebSocket communication. The system is also designed to handle data efficiently by using a combination of MongoDB and MySQL databases.

Several constraints influence the design of the system. The system is implemented using technologies such as Node.js and Express.js, which define the backend structure. The use of two different databases introduces complexity in data synchronization. The system is also constrained by predefined data formats such as marks ranging between 0 and 100.

Constraints:

- Two databases must be available (MongoDB for users, MySQL for marks/attendance).
- Chart.js is loaded from a CDN, requiring outbound internet for UI charts.
- Notifications are stored in memory (not persisted across server restarts).

Other constraints include limited development time, predefined project scope, and the requirement to implement role-based access control. These constraints have influenced the overall design decisions.

3.2 Design Assumptions

The design of the EduTrack system is based on several assumptions that impact its implementation and functionality.

It is assumed that all users have valid login credentials before accessing the system. The system assumes that the databases are properly configured and running at all times. It is also assumed that users interact with the system through a stable internet connection.

Another important assumption is that the data entered by users, such as marks and attendance, follows the correct format and range. The system also assumes that the frontend and backend are properly connected through APIs and that the server is capable of handling multiple concurrent requests.

- MongoDB and MySQL are running on configured hosts before the server starts.
- Users are authenticated using JWT stored in browser localStorage.
- The system is used within a controlled academic environment (classes CS1-CS4; subjects: Mathematics, Physics, English, French, DSA).
- Default seed data is used for demonstration/testing

These assumptions simplify the design and allow the system to focus on its core functionalities.

3.3 Significant Design Packages

The EduTrack system is divided into several significant design packages based on functionality. These packages represent different modules of the system and are organized in a layered structure.

The authentication package handles user login, token generation, and access control. The student data package manages student profiles, attendance records, and related data. The marks entry package is responsible for storing and updating marks. The validation package ensures that all input data is correct before processing.

The notification package handles real-time communication and alert generation. The visualization package processes data and generates graphical representations such as charts.

Backend (Node.js/Express):
- `backend/server.js`: App bootstrap, routes, static hosting, Socket.IO setup.
- `backend/routes/*`: REST endpoints for auth, students, marks.
- `backend/controllers/*`: Request handling and orchestration.
- `backend/modules/*`: Core business logic (auth, student data, marks entry, visualization, notifications, validation).
- `backend/models/*`: MongoDB and MySQL models.

Frontend (Static HTML/CSS/JS):
- `frontend/login.html`: Role-based login.
- `frontend/coordinator-dashboard.html`: Coordinator dashboard and management tools.
- `frontend/teacher-dashboard.html`: Teacher analytics and marks entry.
- `frontend/student-dashboard.html`: Student self-view of performance.

- `frontend/js/charts.js`: Shared chart and validation utilities.

Data Layer:

- MongoDB (users, student profiles).
- MySQL (exam marks, attendance, processed results, subject attendance).

3.4 Dependent External Interfaces

The system depends on several external interfaces for its functionality. These interfaces are used by internal modules to perform operations such as data storage, authentication, and real-time communication.

The table below lists the public interfaces required from external applications.

External Application and Interface Name	Module Using the Interface	Functionality / Description
MongoDB Database	Authentication Module, Student Module	Used to store user credentials and student profiles
MySQL Database	Marks Module, Student Module	Used to store marks, attendance, and processed results
Socket.IO Interface	Notification Module	Used for real-time communication and sending notifications
REST API Interface	Controllers	Used for communication between frontend and backend
Chart.js CDN	Frontend dashboards	Renders charts in browser.

These interfaces enable the system to interact with external components and ensure smooth operation of all functionalities.

3.5 Implemented Application External Interfaces (and SOA web services)

The EduTrack system provides several public interfaces that can be accessed by external applications or the frontend. These interfaces are implemented using RESTful APIs and are handled by specific modules within the system.

The table below lists the implementation of public interfaces provided by the system.

Interface Name	Module Implementing the Interface	Functionality / Description
/api/auth/login	Authentication Module	Handles user login and token generation
/api/students	Student Module	Manages student data (add, update, delete)
/api/marks/update	Marks Module	Updates student marks and triggers result processing
/api/marks/view	Visualization Module	Retrieves and displays performance data
/api/notifications	Notification Module	Sends and manages notifications

Socket.IO Events:
- Client -> Server: register (role, class, subject, studentId)
- Server -> Client: `notification`, `marks_updated`, `class_marks_updated`, `subject_marks_updated`, `attendance_updated`, `class_attendance_updated`

4 Logical View

This section describes the detailed design of the EduTrack Student Performance System. The system is structured in multiple layers, where each layer is responsible for specific functionality and interacts with other layers to complete system operations.

At the highest level, the system consists of the frontend layer, controller layer, module (business logic) layer, and data layer. These layers work together to implement the key use cases such as login, student management, marks entry, and performance visualization.

When a user performs an action, the request is initiated from the frontend and sent to the backend through API calls. The controller layer receives the request and forwards it to the appropriate module. The module processes the request, interacts with the database layer, and returns the response to the controller, which then sends it back to the frontend.

At a more detailed level, each module consists of multiple classes and utilities that collaborate to perform specific tasks. For example, the marks module includes classes for storing marks, calculating results, and updating processed data. These classes interact with validation utilities and database models to ensure correct functionality.

This layered decomposition ensures clear separation of responsibilities and improves system maintainability and scalability.

4.1 Design Model

The design model of the EduTrack system is based on modular decomposition, where the system is divided into multiple modules, and each module consists of significant classes responsible for specific functionalities.

1. Authentication Module Classes:

- User
- Authentication Controller

Responsibilities:

The authentication module handles user login and access control. It verifies user credentials and generates secure tokens.

Attributes:

- username
- password
- role **Operations:**
- login()
- verifyUser()
- generateToken()

Relationships:

Interacts with the database to validate user credentials and with controllers to process login requests.

2. Student Data Module

Classes:

- StudentProfile
- StudentController

Responsibilities:

Manages student information, including adding, updating, and deleting student records.

Attributes:

- studentId
- name
- class
- attendance **Operations:**
- addStudent()
- updateStudent()
- deleteStudent()

Relationships:

Interacts with both MongoDB (for profiles) and MySQL (for attendance records).

3. Marks Module Classes:

- ExamMark
- MarksController

- ProcessedResult

Responsibilities:
Stores and processes student marks, calculates averages, and assigns grades.

Attributes:

- subject
- marks
- average
- grade **Operations:**
- updateMarks()
- calculateAverage()
- assignGrade()

Relationships:
Works with validation module and database tables to ensure correct data storage and processing.

4. Validation Module

Classes:

- ValidationUtility

Responsibilities:
Validates input data such as marks, student details, and login credentials.

Operations:

- validateMarks()
- validateInput()

5. Notification Module

Classes:

- NotificationService()

Responsibilities:
Handles sending real-time notifications to users.

Operations:

- sendNotification()
- broadcastMessage()

6. Visualization Module

Classes:

- VisualizationService

Responsibilities:
Generates data for charts and graphical representations.

Operations:

- generateChartData()

- getPerformanceMetrics()

4.2 Use Case Realization

This section describes how each use case defined in Section 2 is implemented within the system design.

Use Case Realization: User Login

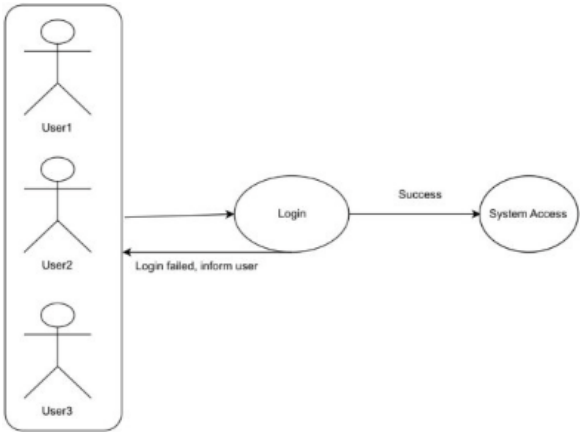
At the highest level, the login process starts from the frontend where the user enters credentials. The request is sent to the authentication controller via an API call. The controller calls the authentication module to verify the credentials.

The module interacts with the database to check if the user exists. If valid, a token is generated and returned to the frontend. The user is then redirected to the dashboard.

Interaction Flow:

Frontend → API → Controller → Authentication Module → Database → Response → Frontend

Case Diagram :



Use Case Realization: Add Student

The coordinator submits student details through the frontend. The request is sent to the student controller, which validates the input using the validation module.

After validation, the student module stores the data in MongoDB and initializes related records in MySQL. A response is sent back confirming successful addition.

Interaction Flow:

Frontend → Controller → Validation Module → Student Module → Database → Response

Use Case Realization: Update Marks

The teacher enters marks through the frontend interface. The request is sent to the marks controller, which invokes the validation module to verify the marks.

Once validated, the marks module stores the data in the database and calculates the average and grade. The notification module sends updates to users.

Interaction Flow:

Frontend → Controller → Validation → Marks Module → Database → Notification Module → Response

Use Case Realization: View Performance

The student requests performance data from the dashboard. The request is processed by the visualization module, which retrieves data from the database and generates chart data.

The processed data is sent back to the frontend for display.

Interaction Flow:

Frontend → Controller → Visualization Module → Database → Response → Frontend

5 Data View

This section describes the persistent data storage structure of the EduTrack Student Performance System. The system uses a combination of databases to efficiently manage different types of data. MongoDB is used for storing flexible data such as user credentials and student profiles, while MySQL is used for structured data such as marks, attendance, and processed results.

The data view explains how data is organized, stored, and accessed within the system. It also defines the relationships between different data entities and how they interact with each other to support system functionality.

5.1 Domain Model

The domain model represents the key entities in the system and the relationships between them. These entities correspond to real-world objects such as users, students, marks, and attendance.

The main entities in the EduTrack system include:

- User: Represents system users such as coordinators, teachers, and students.
- StudentProfile: Represents student information including name, class, and other details.
- ExamMark: Represents marks obtained by students in different subjects and exams.
- Attendance: Represents student attendance records.
- ProcessedResult: Represents computed data such as average marks and grades. Relationships:
- A User can be associated with one role (Coordinator, Teacher, or Student).
- A StudentProfile is linked to a User account.
- A StudentProfile has multiple ExamMark records.
- A StudentProfile has one Attendance record.
- A StudentProfile has one ProcessedResult record.

Relationships:

- One User can map to one StudentProfile (student role).
- One StudentProfile has many ExamMark rows (subject ? exam).
- One StudentProfile has one Attendance row.
- One StudentProfile has many SubjectAttendance rows.
- One StudentProfile has one ProcessedResult row.

These relationships ensure that all data related to a student is interconnected and easily accessible.

5.2 Data Model (persistent data view)

The data model describes how the domain entities are physically stored in databases. The EduTrack system uses two types of databases:

MongoDB Collections:

- **Users Collection:** Stores user credentials such as username, password, and role.
- **StudentProfiles Collection:** Stores student personal details.

MySQL Tables:

- **ExamMark Table:** Stores subject-wise marks for students.
- **Subject Attendance Table:** Stores attendance percentage of students.
- **ProcessedResult Table:** Stores computed results such as average marks and grades.

The separation of data into two databases improves performance and scalability. MongoDB is used for flexible data storage, while MySQL ensures structured and relational data management.

5.2.1 Data Dictionary

the data dictionary provides detailed information about each data element used in the system.

User Table / Collection

Field Name	Data Type	Description
userId	String	Unique identifier for each user
username	String	Login username
password	String	Encrypted password
User role	String	Role of user (Coordinator, Teacher, Student)

StudentProfile Table / Collection

Field Name	Data Type	Description
studentId	String	Unique identifier for student
name	String	Student name
class	String	Class or section
userId	String	Linked user account

ExamMark Table

Field Name	Data Type	Description
Mark Id	Integer	Unique identifier for mark record
studentId	String	Student reference
subject	String	Subject name
exam1	Integer	Marks in exam 1
exam2	Integer	Marks in exam 2
exam3	Integer	Marks in exam 3
exam4	Integer	Marks in exam 4

Attendance Table

Field Name	Data Type	Description
attendanceId	Integer	Unique identifier
studentId	String	Student reference
percentage	Float	Attendance percentage

ProcessedResult Table

Field Name	Data Type	Description
resultId	Integer	Unique identifier
studentId	String	Student reference
average	Float	Average marks
grade	String	Grade assigned

6 Exception Handling

This section describes how exceptions are handled in the EduTrack Student Performance System. Exception handling ensures that the system can manage errors effectively without crashing and provides meaningful feedback to users.

The system is designed to detect, handle, and log errors that occur during execution. Exceptions may arise due to invalid input, authentication failures, database errors, or unauthorized access attempts. Each exception is handled at the appropriate level to maintain system stability and security.

6.1 Types of Exceptions

The following are the main types of exceptions handled by the system:

1. Authentication Exceptions

These exceptions occur when there is an issue with user login or access control.

- **Cause:** Invalid username or password
- **Handling:** The system verifies credentials and returns an error message if authentication fails
- **Follow-up Action:** User is prompted to re-enter valid credentials

2. Authorization Exceptions

These occur when a user tries to perform an action without proper permissions.

- **Cause:** Unauthorized access to restricted APIs
- **Handling:** The system checks user roles using tokens and denies access
- **Follow-up Action:** Error message displayed indicating insufficient permissions

3. Validation Exceptions

These exceptions occur when user input does not meet the required conditions.

- **Cause:** Marks outside valid range (0–100), missing fields, incorrect data format
- **Handling:** Input is validated before processing and invalid data is rejected • **Follow-up Action:** User is asked to correct the input

4. Database Exceptions

These occur when there is an issue with database operations.

- **Cause:** Connection failure, query errors, missing records
- **Handling:** Errors are caught at the database interaction layer and appropriate messages are returned
- **Follow-up Action:** Retry operation or notify administrator if the issue persists

5. Not Found Exceptions

These occur when requested data is not available.

- **Cause:** Student record or mark not found
- **Handling:** The system checks for data existence before processing
- **Follow-up Action:** Inform the user that the requested data does not exist

6. Server Exceptions

These are unexpected errors that occur during system execution.

- **Cause:** Internal server issues or unhandled exceptions
- **Handling:** The system catches such errors and returns a generic error response
- **Follow-up Action:** Error is logged for debugging and system maintenance

6.2 Exception Logging

All exceptions are logged to help in debugging and monitoring system behavior. Logging is performed at the backend server level.

The logs include:

- Type of error
- Time of occurrence
- Module where the error occurred

- Error message

These logs help developers identify issues and improve system reliability.

6.3 Error Response Mechanism

The system returns appropriate HTTP status codes along with error messages:

- **400 (Bad Request):** Missing or Invalid input data
- **401 (Unauthorized):** Missing JWT Token or Authentication failure
- **403 (Forbidden):** Access denied (Role not found or invalid tokens)
- **404 (Not Found):** Requested data not found (e.g Student ID or user not found)
- **500 (Internal Server Error):** Unexpected server error or Database failures

This ensures that the frontend can properly handle errors and display meaningful messages to users.

6.4 Follow-up Actions

After an exception occurs, the system ensures that proper actions are taken:

- Users are informed with clear and understandable error messages
- Invalid operations are prevented from affecting system data
- Critical errors are logged for further analysis
- The system continues functioning without interruption

7 Configurable Parameters

This section describes the configurable parameters used in the EduTrack Student Performance System. Configurable parameters are system settings that can be modified to control application behavior without changing the source code.

These parameters are typically stored in environment variables or configuration files and are used to define database connections, security settings, and application behavior. Some parameters are dynamic, meaning they can be modified without restarting the system, while others require a system restart to take effect.
The use of configurable parameters improves flexibility, maintainability, and adaptability of the system.

Simple Configurable Parameters

The table below describes the key configuration parameters used in the system along with their definitions and whether they are dynamic.

	Definition and Usage	**Dynamic?**
JWT_SECRET	Secret key used for generating and verifying authentication tokens. Ensures secure user authentication.	No
MONGO_DB_URL	Connection string used to connect to the MongoDB database for storing user and student profile data.	No

MYSQL_HOST	Specifies the host address of the MySQL database server.	No
MYSQL_USER	Username used to authenticate with the MySQL database.	No
MYSQL_PASSWORD	Password used for MySQL database access.	No
MYSQL_DATABASE	Name of the MySQL database used by the application.	No
PORT	Specifies the port number on which the backend server runs.	No
NODE_ENV	Defines the environment mode (development or production). Affects logging and error handling.	No
MAX_MARKS_LIMIT	Defines the maximum allowed marks value for validation (e.g., 100).	Yes
MIN_MARKS_LIMIT	Defines the minimum allowed marks value (e.g., 0).	Yes
NOTIFICATION_THRESHOLD	Defines threshold for sending alerts (e.g., low performance or attendance).	Yes

The database-related parameters such as MongoDB and MySQL connection details are critical for system operation and require restarting the application if changed. Security-related parameters like JWT_SECRET also require a restart because they affect authentication mechanisms.
On the other hand, validation-related parameters such as marks limits and notification thresholds can be configured dynamically, allowing the system to adapt to changing requirements without downtime.

8 Quality of Service

This section describes the non-functional aspects of the EduTrack Student Performance System, including availability, security, performance, and monitoring. These aspects ensure that the system operates reliably, securely, and efficiently under different conditions.

The design of the system incorporates mechanisms to support continuous operation, secure access, efficient data processing, and effective monitoring of system behavior.

8.1 Availability

The EduTrack system is designed to provide high availability so that users can access the system at any time with minimal interruptions. The system runs on a backend server developed using Node.js and Express.js, which ensures continuous request handling.

The application depends on the availability of both databases, MongoDB and MySQL. Proper database connectivity is essential for system operation. If either database becomes unavailable, certain functionalities may be affected.

Periodic maintenance activities such as database updates or server restarts may temporarily impact availability. However, these activities are planned during low-usage periods to minimize disruption. The system is also designed to handle minor failures gracefully without crashing.

8.2 Security and Authorization

Security is a critical aspect of the EduTrack system. The system implements secure authentication and authorization mechanisms to protect user data and restrict access to authorized users only.
User authentication is handled using JSON Web Tokens (JWT). When a user logs in, a secure token is generated and used to verify all subsequent requests. This ensures that only authenticated users can access protected resources.
The system also implements role-based access control. Different users such as coordinators, teachers, and students have different permissions. For example, only coordinators can add or delete students, while teachers can update marks and students can only view their performance.
Sensitive data such as passwords are securely stored in encrypted form. Unauthorized access attempts are blocked, and appropriate error responses are returned. TYPES :
- Authentication uses JWT (8-hour expiry).
- Passwords are hashed with bcrypt.
- Authorization is role-based in route middleware.
- Sensitive endpoints require valid JWT.
- CORS is enabled for all origins (can be tightened for production).

8.3 Load and Performance Implications

The EduTrack system is designed to handle multiple users and requests efficiently. Performance optimization is achieved through efficient database design and use of caching mechanisms.
The system uses a ProcessedResult table to store precomputed results such as average marks and grades. This reduces the need for repeated calculations and improves response time when displaying performance data. The system is capable of handling multiple concurrent requests, such as simultaneous login requests or marks updates. Efficient API design and modular architecture ensure that each request is processed quickly. Database growth is expected as more student records and marks are added over time. Proper indexing and optimized queries are used to maintain performance even as data volume increases. - MySQL processed results cache reduces heavy aggregation queries.
- Chart data is returned pre-aggregated for fast UI rendering.
- WebSockets minimize polling and improve responsiveness.
- System is designed for classroom-scale usage (CS1-CS4).

These design considerations ensure that the system meets performance requirements and can support load increases.

8.4 Monitoring and Control

The EduTrack system includes mechanisms for monitoring system activity and controlling operations. The backend server logs important events such as API requests, errors, and database operations.

These logs include details such as timestamps, error messages, and module information, which help in diagnosing issues and improving system reliability.

The system also uses real-time communication through Socket.IO to monitor and notify users about important events such as marks updates and attendance alerts.

Controllable processes in the system include API request handling, database operations, and notification services. These processes can be monitored and managed to ensure smooth system operation.

Monitoring helps administrators identify performance issues, detect failures, and take corrective actions to maintain system stability.

- Server logs connection status for MongoDB/MySQL.
- Errors are logged to console for troubleshooting.
- No external monitoring tool is configured by default.